Assignment - 1

CS4186 Computer Vision

Utkarsh JAIN
55992915
21 April 2022

Supporting Functions

I would like to talk about some supporting functions that I am using to load images and the function I run before running retrieval methods.

I have 2 functions that iterate over all availble files in a directory and return the images in a numpy array. Instead of loading all the images in memory, Python's generators have been used using the '`yield`' keyword. It returns the img in a numpy array and stops execution and when the next image is requested it resumes the function right where it was left during last execution. This is done by saving the process information.

I've used multi-threading with opencv. Opencv uses suspectedly uses multi-processing so even with Python's dreaded Global Interpreter Lock (GIL), all processor cores are used.

Secondly in my multi-threaded retrieval functions I have used a decorator `@check_per_query_existence_and_display` & `@check_queries_existence_and_display` which wraps the multi-threaded functions with extra code. This extra code checks a disk-stored pickle file if the results for this particular query are already present. If they are present instead of running the entire computation it falls back to the store results.

One more thing to note is that code is very modular allowing for swift modifications without any problem.

Sequence diagrams used in the three diagrams demonstrate the situation when the code is finding features instead of finding it on disk.

Using multithreading features were stored on disk.

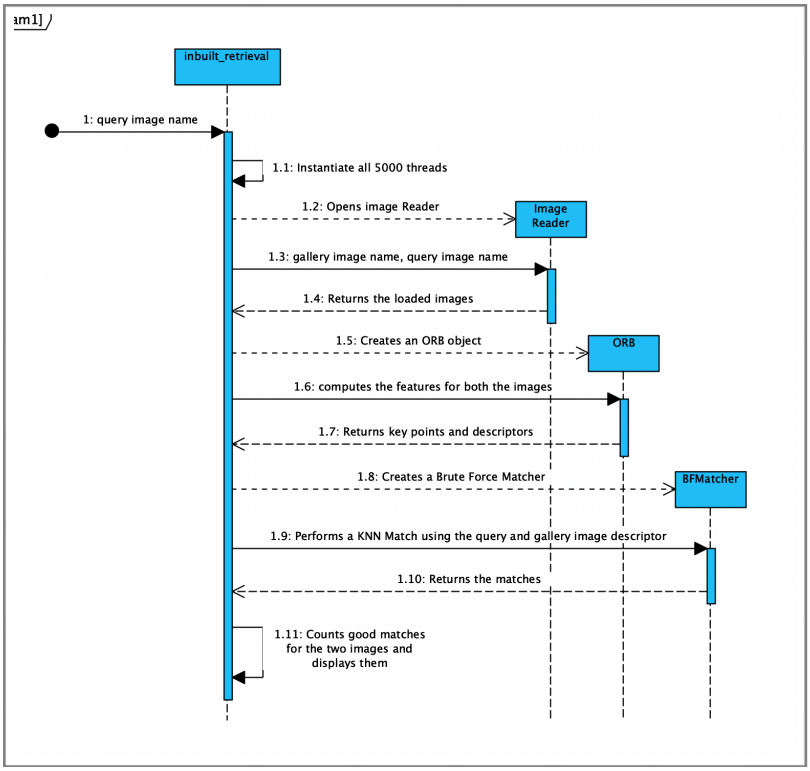
Method 1

I am using ORB method to extract features and a brute force match features generated by ORB. I decided to not use SIFT because it's computationally demanding (also taking a lot of time) not effective for low powered devices. SURF is way better than SIFT in runtime and robustness but ORB is faster than SURF.

Flow of code is that at first a per query image is performed then and then all 5000 computations are given their own thread. Because of python's GIL it was expected to run on a single core with a lot of thrashing but it ran on all cpu cores, reducing the runtime. The code flow is demonstrated below. For this method parameters have been fine tuned for all the query image to optimise the results. Number of features that the orb object considers and

the distance ratio for matches produced by Knn match has been regulated for all query images. Optimum parameter are stored on disk and recovered as & when required. It's worth noting that features are not extracted everytime, a copy of the extracted features is stored on disk and read as & when required.

It does very well for some query images and fails for some. It has horrible results for query 1 but for the second last query it finds 10 matching images. I was surprising that some images do better with comparatively lesser features. You can see it fails when the lighting, perspective & distance from the object is changed.



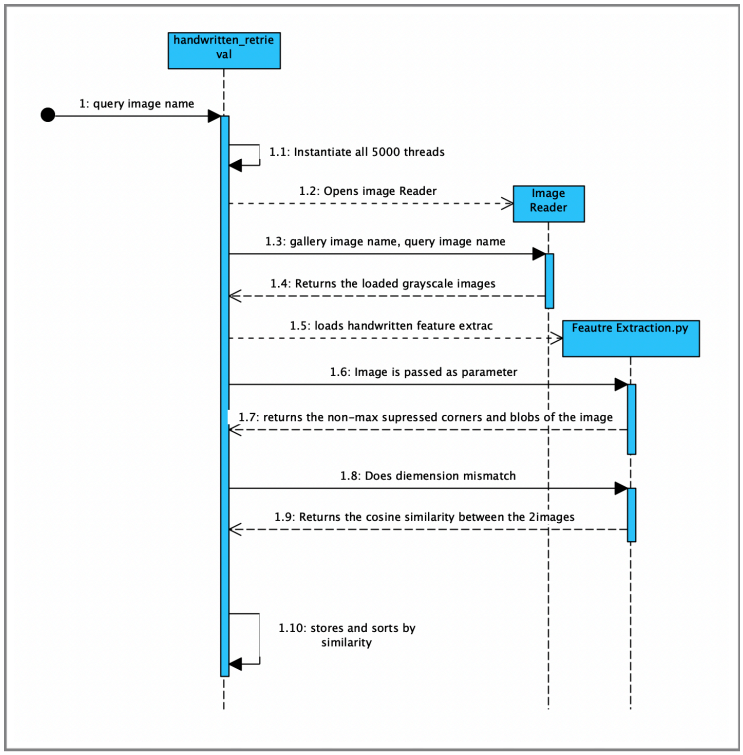
Flow for inbuilt method

```
Q1:3457.jpg 4463.jpg 2718.jpg 4684.jpg 3644.jpg 1077.jpg 4142.jpg 3320.jpg 1646.jpg 2840.jpg
Q2:86.jpg 930.jpg 1860.jpg 2082.jpg 4249.jpg 3544.jpg 4523.jpg 1238.jpg 630.jpg 1192.jpg
Q3:3113.jpg 3399.jpg 2684.jpg 4748.jpg 3974.jpg 1592.jpg 2015.jpg 83.jpg 1169.jpg 4118.jpg
Q4:2575.jpg 1799.jpg 173.jpg 1323.jpg 4918.jpg 2427.jpg 4456.jpg 2656.jpg 2473.jpg 4772.jpg
Q5:2403.jpg 4937.jpg 3065.jpg 1159.jpg 3592.jpg 4577.jpg 4371.jpg 538.jpg 2570.jpg 4056.jpg
Q6:2003.jpg 1227.jpg 991.jpg 839.jpg 934.jpg 2490.jpg 3875.jpg 309.jpg 4748.jpg 3743.jpg
Q7:2857.jpg 3005.jpg 2726.jpg 79.jpg 2701.jpg 2402.jpg 2683.jpg 1603.jpg 2444.jpg 3628.jpg
Q8:770.jpg 4277.jpg 3660.jpg 426.jpg 2390.jpg 4986.jpg 3342.jpg 1913.jpg 3476.jpg 221.jpg
Q9:2666.jpg 540.jpg 1750.jpg 3086.jpg 4991.jpg 880.jpg 4537.jpg 2711.jpg 3342.jpg 2172.jpg
Q10:4018.jpg 799.jpg 4064.jpg 3884.jpg 3947.jpg 3655.jpg 1279.jpg 139.jpg 4151.jpg 2842.jpg
Q11:1044.jpg 4386.jpg 2870.jpg 1352.jpg 3362.jpg 3791.jpg 3068.jpg 1055.jpg 1370.jpg 820.jpg
Q12:4256.jpg 3188.jpg 1130.jpg 2966.jpg 3268.jpg 3908.jpg 4967.jpg 670.jpg 3448.jpg 1841.jpg
Q13:3331.jpg 990.jpg 4576.jpg 584.jpg 4228.jpg 3370.jpg 739.jpg 3493.jpg 4569.jpg 1954.jpg
Q14:4325.jpg 810.jpg 291.jpg 3166.jpg 716.jpg 2859.jpg 3481.jpg 4978.jpg 4823.jpg 1213.jpg
Q15:1038.jpg 1277.jpg 1368.jpg 3255.jpg 4027.jpg 4467.jpg 3457.jpg 2354.jpg 3048.jpg 4630.jpg
Q16:456.jpg 540.jpg 2853.jpg 4537.jpg 3255.jpg 710.jpg 793.jpg 83.jpg 3681.jpg 4991.jpg
Q17:2.jpg 3581.jpg 3753.jpg 3255.jpg 1229.jpg 3327.jpg 1716.jpg 2626.jpg 1839.jpg 710.jpg
Q18:1276.jpg 649.jpg 1164.jpg 1710.jpg 4792.jpg 116.jpg 862.jpg 756.jpg 3956.jpg 3342.jpg
Q19:4286.jpg 4184.jpg 1784.jpg 4332.jpg 2807.jpg 518.jpg 1884.jpg 1459.jpg 4170.jpg 1028.jpg
Q20:3259.jpg 672.jpg 2389.jpg 1876.jpg 411.jpg 3107.jpg 4389.jpg 470.jpg 4009.jpg 1499.jpg
```

Method 2

I struggled very hard to find a SIFT/SURF or ORB online. These are complicated algorithms and to find one implemented online was difficult. Of

course there are many implementations online so why was it difficult for me? I wanted to understand every bit and piece of the algorithm and these complex algorithms don't make it any easier. Although professor did teach SIFT in detail in class, the implementation still had holes and I couldn't get through. I found that Harris corner matching and blob was easier and I could implement it. I was able to extract decent corners & blobs and then decided to use cosine similarity. I came across an error while trying to use cosine similarity, that is because for different images feature arrays are of different sizes and cosine similarity will only work for images of the same size. What I did was, I wrote a function that slides the smaller image over the bigger image pixel by pixel and calculates cosine similarity between the smaller image and the part of the bigger image that the smaller image overlaps in that iteration. It returns the maximum cosine similarity between these 2 images. Unfortunately I wasn't able to combine the blob extractor with Harris corner detector. The blob detector works feel free to use it. This output from corner information matched with the query image..



Flow for method 2

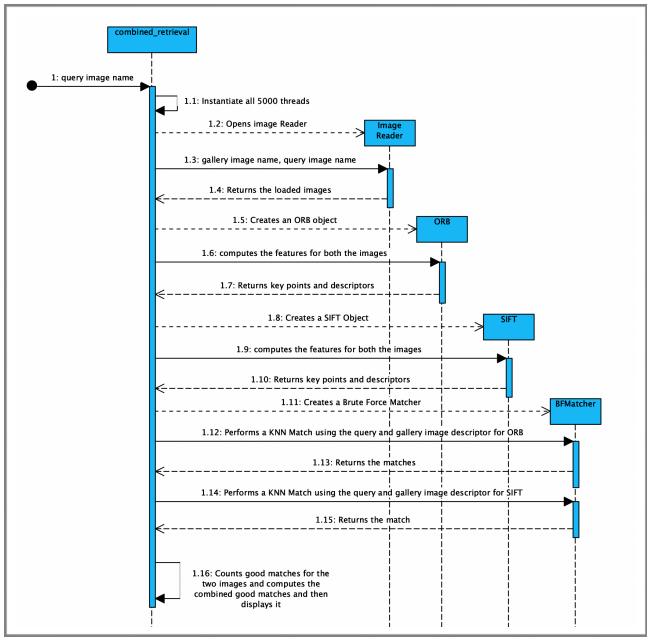
```
Q1:4066.jpg 3470.jpg 4934.jpg 4075.jpg 428.jpg 113.jpg 4001.jpg 1144.jpg 448.jpg 99.jpg
Q2:4934.jpg 4399.jpg 448.jpg 4066.jpg 882.jpg 3470.jpg 4214.jpg 4981.jpg 2200.jpg 2662.jpg
Q3:4066.jpg 4001.jpg 3470.jpg 1381.jpg 4791.jpg 2606.jpg 665.jpg 4766.jpg 1203.jpg 4568.jpg
Q4:4001.jpg 2970.jpg 1919.jpg 4809.jpg 4066.jpg 2522.jpg 4519.jpg 448.jpg 2198.jpg 1364.jpg
Q5:4001.jpg 3470.jpg 140.jpg 448.jpg 4619.jpg 2198.jpg 634.jpg 3247.jpg 4191.jpg 3877.jpg
Q6:448.jpg 4191.jpg 4066.jpg 1768.jpg 4001.jpg 3247.jpg 1432.jpg 791.jpg 88.jpg 357.jpg
Q7:3470.jpg 4066.jpg 4001.jpg 533.jpg 1919.jpg 2429.jpg 2522.jpg 634.jpg 3247.jpg 2198.jpg
Q8:428.jpg 4075.jpg 4001.jpg 3470.jpg 448.jpg 3247.jpg 533.jpg 1144.jpg 495.jpg 3877.jpg
Q9:4066.jpg 4998.jpg 198.jpg 3950.jpg 1831.jpg 128.jpg 2796.jpg 1951.jpg 3895.jpg 1555.jpg
Q10:4934.jpg 4066.jpg 448.jpg 3470.jpg 2662.jpg 882.jpg 4214.jpg 2815.jpg 461.jpg 2429.jpg
Q11:4075.jpg 428.jpg 4001.jpg 448.jpg 2970.jpg 3561.jpg 4934.jpg 3470.jpg 4066.jpg 461.jpg
Q12:3470.jpg 448.jpg 4066.jpg 4934.jpg 4001.jpg 2522.jpg 2970.jpg 2198.jpg 533.jpg 4619.jpg
Q13:2429.jpg 4001.jpg 2862.jpg 128.jpg 4663.jpg 4809.jpg 462.jpg 2796.jpg 3195.jpg 4191.jpg
Q14:4001.jpg 2429.jpg 3470.jpg 448.jpg 3561.jpg 2522.jpg 4191.jpg 462.jpg 1864.jpg 128.jpg
Q15:4934.jpg 882.jpg 4798.jpg 4191.jpg 2862.jpg 461.jpg 1575.jpg 4001.jpg 3051.jpg 368.jpg
Q16:4934.jpg 1691.jpg 3177.jpg 2730.jpg 3027.jpg 2558.jpg 2974.jpg 2566.jpg 3814.jpg 1500.jpg
Q17:4066.jpg 4934.jpg 448.jpg 533.jpg 4399.jpg 882.jpg 428.jpg 1864.jpg 4075.jpg 4001.jpg
Q18:4066.jpg 448.jpg 3247.jpg 4001.jpg 3470.jpg 3561.jpg 428.jpg 3877.jpg 4075.jpg 4191.jpg
Q19:4066.jpg 448.jpg 3470.jpg 3247.jpg 2522.jpg 4001.jpg 1919.jpg 3561.jpg 4191.jpg 533.jpg
Q20:448.jpg 3247.jpg 1144.jpg 533.jpg 1941.jpg 3470.jpg 3877.jpg 908.jpg 557.jpg 53.jpg
```

This method is not rotation invariant and only partially scale invariant.

* I took internet's help to implement both Harris corner detector and blob detector.

Method 3 - Combined

I've tried to combine SIFT with ORB. What I found was that there was no improvement in performance, infact the overall accuracy seemed to have dropped. My Hypothesis is that SIFT is worse than ORB and when we combine SIFT & ORB we don't get better results. My combination strategy was to create a combined score from both the methods using the formula K1*K2 / (K1 + K2) where K1 is the good matches from ORB algorithm and K2 is the number of good matches from SIFT Algorithm. Good Matches is number of matches found within some specified distance. One more thing to note was that if I try to store the features on a file SIFT features are more expensive to keep on disk, that's because uses a vector of size 128, whereas ORB uses a vector of size 32. Results are below




```
Q1:3457.jpg 3743.jpg 4467.jpg 2450.jpg 839.jpg 4276.jpg 1137.jpg 890.jpg 290.jpg 1788.jpg
Q2:86.jpg 930.jpg 2082.jpg 1860.jpg 3544.jpg 1238.jpg 4249.jpg 630.jpg 4523.jpg 1192.jpg
Q3:3113.jpg 3399.jpg 4442.jpg 2684.jpg 4987.jpg 1592.jpg 452.jpg 4952.jpg 1436.jpg 4871.jpg
Q4:2575.jpg 2427.jpg 4918.jpg 173.jpg 1799.jpg 1083.jpg 1323.jpg 4067.jpg 4456.jpg 2473.jpg
Q5:2403.jpg 4937.jpg 3857.jpg 3065.jpg 1227.jpg 4871.jpg 4502.jpg 3166.jpg 1159.jpg 2490.jpg
Q6:2003.jpg 1227.jpg 934.jpg 991.jpg 839.jpg 2490.jpg 3875.jpg 2935.jpg 427.jpg 3092.jpg
Q7:2857.jpg 3005.jpg 2726.jpg 79.jpg 1603.jpg 2683.jpg 2701.jpg 2402.jpg 2444.jpg 1234.jpg
Q8:770.jpg 3599.jpg 426.jpg 3428.jpg 2050.jpg 2999.jpg 3660.jpg 4277.jpg 216.jpg 644.jpg
Q9:2666.jpg 4991.jpg 1750.jpg 3086.jpg 966.jpg 540.jpg 4079.jpg 4276.jpg 4138.jpg 3991.jpg
Q10:4018.jpg 2568.jpg 3884.jpg 1722.jpg 667.jpg 1634.jpg 2666.jpg 1794.jpg 1792.jpg 2115.jpg
Q11:1044.jpg 4386.jpg 2870.jpg 1352.jpg 3791.jpg 3362.jpg 4794.jpg 921.jpg 4977.jpg 4632.jpg
Q12:4256.jpg 3188.jpg 2966.jpg 3268.jpg 4871.jpg 2399.jpg 1426.jpg 670.jpg 3029.jpg 1906.jpg
Q13:3331.jpg 990.jpg 584.jpg 4228.jpg 4576.jpg 3493.jpg 3370.jpg 739.jpg 4569.jpg 769.jpg
Q14:4325.jpg 3166.jpg 810.jpg 3743.jpg 291.jpg 2859.jpg 3092.jpg 716.jpg 4978.jpg 839.jpg
Q15:1038.jpg 3577.jpg 1481.jpg 4415.jpg 4506.jpg 3854.jpg 1270.jpg 2458.jpg 1969.jpg 422.jpg
Q16:456.jpg 4871.jpg 1520.jpg 2425.jpg 3255.jpg 525.jpg 3029.jpg 1264.jpg 1634.jpg 3166.jpg
Q17:2.jpg 1481.jpg 2397.jpg 4467.jpg 852.jpg 2859.jpg 4871.jpg 3092.jpg 1249.jpg 3595.jpg
Q18:1276.jpg 649.jpg 1164.jpg 1710.jpg 4792.jpg 715.jpg 116.jpg 501.jpg 862.jpg 3368.jpg
Q19:4286.jpg 1784.jpg 4332.jpg 518.jpg 4184.jpg 2807.jpg 1884.jpg 1459.jpg 4170.jpg 1028.jpg
Q20:3259.jpg 672.jpg 1876.jpg 2389.jpg 534.jpg 1215.jpg 2437.jpg 1270.jpg 298.jpg 227.jpg
```